

Final Report:

FPGA Implementation and Official BeagleV-Fire Gateway Integration of a Denoise Accelerator

Yicheng Wang

Student ID: 123090586

May 3, 2026

Abstract

This final report continues the hardware–software co-design project for accelerating the denoise stage of an Image Signal Processing (ISP) pipeline on the BeagleV-Fire RISC-V + FPGA platform. The midterm stage completed the RTL-level design and simulation verification of the denoise accelerator, including the denoise core, AXI4-Stream wrapper, FIFO-based backpressure handling, and controlled wrapper with `start`, `busy`, and `done` signals.

The focus of this final stage is not to repeat the RTL simulation work, but to move toward FPGA implementation and board-level integration. First, the verified RTL design was imported into Libero SoC 2025.2 and implemented as a standalone FPGA design targeting the PolarFire SoC device used by BeagleV-Fire. Synthesis, place-and-route, resource analysis, and timing analysis were completed successfully. The standalone accelerator used limited FPGA resources and achieved an estimated maximum frequency of approximately 196.85 MHz.

However, a standalone FPGA module is not sufficient for actual BeagleV-Fire execution because it does not include the RISC-V MSS subsystem, DDR interface, Linux boot support, device-tree overlay, DMA path, or software-accessible control interface. Therefore, the second part of this report focuses on reproducing the official BeagleV-Fire gateway build flow under Windows. Multiple compatibility issues were identified and fixed, including tool path configuration, HSS generation problems, Windows path handling, Libero script argument truncation, temporary file permission issues, and a Tcl flow interruption after place-and-route.

After these fixes, the official default BeagleV-Fire gateway was successfully built on Windows, exported as a LinuxProgramming SPI bitstream, transferred to the board, and deployed using the official Linux-side gateway update mechanism. This confirmed that the patched Windows build flow could produce a usable board-level bitstream. Finally, this report discusses the CAPE / VERILOG_TEMPLATE / APB route as a safer first step for custom accelerator integration, analyzes the deployment issue encountered during an experimental APB-test gateway update, and proposes a safer revised integration plan.

Contents

1	Introduction	5
2	Standalone FPGA Implementation in Libero SoC	6
2.1	Purpose of Standalone Implementation	6
2.2	Libero Environment	6
2.3	Synthesis Result	6
2.4	SmartDesign View	7
3	Resource Utilization and Timing Analysis	8
3.1	Resource Utilization	8
3.2	Timing Result	8
4	Why Standalone Implementation Is Not Enough	9
5	Official BeagleV-Fire Gateware Build Flow	10
5.1	Project Directory and Build Command	10
5.2	Toolchain Setup	10
6	Windows Compatibility Issues and Fixes	11
6.1	Path and Tool Problems	11
6.2	Python Output Decoding	11
6.3	HSS and Configuration Generation	12
6.4	MSS PermissionError	12
6.5	Libero SCRIPT_ARGS Path Truncation	12
6.6	Tcl Temporary File Permission Issue	12
6.7	Two-Stage Libero Flow	12
6.8	Summary	13
7	Default Bitstream Generation and Board Validation	14
7.1	Successful Default Bitstream Export	14
7.2	Board Access and File Transfer	15
7.3	Linux-Side Gateware Update	15
8	CAPE / VERILOG_TEMPLATE / APB Integration Investigation	16
8.1	Why APB First	16
8.2	VERILOG_TEMPLATE	16
8.3	APB Register Test	16
8.4	APB Test Build	17
8.5	Experimental Deployment	17

9	Deployment Issue and Risk Analysis	18
9.1	Observed Issue	18
9.2	Likely Cause	18
9.3	Recovery Options	19
9.4	Lessons Learned	19
10	Revised Integration Plan	19
10.1	Step 1: Restore Default Gateway	19
10.2	Step 2: Use Matched LinuxProgramming Files	20
10.3	Step 3: Verify APB Register Access	20
10.4	Step 4: Connect Denoise Control Signals	20
10.5	Step 5: Add Data Movement	20
11	Conclusion	21

1 Introduction

This project aims to accelerate the denoise stage of an Image Signal Processing (ISP) pipeline on the BeagleV-Fire platform. The earlier stages of the project already showed that denoise is the main computational bottleneck in the software pipeline. Based on that result, the midterm stage implemented and verified a streaming RTL denoise accelerator.

The verified RTL design includes a 3×3 Gaussian-like denoise core, AXI4-Stream input and output interfaces, FIFO-based backpressure support, and a controlled wrapper with **start**, **busy**, and **done** signals. The controlled wrapper successfully processed one 64×64 input frame and generated 3844 valid output pixels.

Figure 1 briefly shows the final RTL simulation result from the midterm stage. This figure is included only as a transition point. The main focus of this final report is the next step: FPGA implementation and official BeagleV-Fire gateway integration.

```
PS E:\DDA2081\fpga_isp_denoise> iverilog -g2012 -o sim_controlled.vvp denoise_core_simple.sv denoise_axis_stream_simple.sv
sv denoise_controlled_axis.sv:77: warning: System task ($display) cannot be synthesized in an always_ff process.
tb_denoise_controlled_axis.sv:82: warning: System task ($display) cannot be synthesized in an always_ff process.
tb_denoise_controlled_axis.sv:87: warning: System task ($fdisplay) cannot be synthesized in an always_ff process.
PS E:\DDA2081\fpga_isp_denoise> vvp sim_controlled.vvp
VCD info: dumpfile wave_controlled_axis.vcd opened for output.
[OK] CONTROLLED output[0][0] = 160
[OK] CONTROLLED output[0][1] = 159
[OK] CONTROLLED output[0][2] = 161
[OK] CONTROLLED output[0][3] = 164
[OK] CONTROLLED output[0][4] = 162
[OK] CONTROLLED output[0][5] = 161
[OK] CONTROLLED output[0][6] = 170
[OK] CONTROLLED output[0][7] = 181
[OK] CONTROLLED output[0][8] = 160
[OK] CONTROLLED output[0][9] = 92
[OK] CONTROLLED output[0][10] = 32
[OK] CONTROLLED output[0][11] = 15
[OK] CONTROLLED output[0][12] = 15
[OK] CONTROLLED output[0][13] = 14
[OK] CONTROLLED output[0][14] = 14
[OK] CONTROLLED output[0][15] = 14
[OK] CONTROLLED output[0][16] = 14
[OK] CONTROLLED output[0][17] = 14
[OK] CONTROLLED output[0][18] = 14
[OK] CONTROLLED output[0][19] = 20
=====
CONTROLLED AXIS TEST PASSED
Output count = 3844
busy=0 done=1
=====
tb_denoise_controlled_axis.sv:187: $finish called at 41150000 (lps)
PS E:\DDA2081\fpga_isp_denoise> cmd /c fc expected_py.hex output_controlled_hw.hex
正在比较文件 expected_py.hex 和 OUTPUT_CONTROLLED_HW.HEX
Fc: 找不到差异
```

Figure 1: Final controlled AXI4-Stream simulation result from the midterm stage.

The work in this final stage answers the following questions:

- Can the verified RTL accelerator be synthesized and implemented in Libero SoC?
- What are the resource and timing results of the standalone FPGA design?
- Why can the standalone design not be directly used as a BeagleV-Fire board image?
- How can the official BeagleV-Fire gateway build flow be made to work under Windows?
- Can a Windows-generated default bitstream be deployed successfully on the board?
- What is a safer route for integrating custom denoise logic into the official gateway?

2 Standalone FPGA Implementation in Libero SoC

2.1 Purpose of Standalone Implementation

Before modifying the official BeagleV-Fire gateway, the verified RTL accelerator was first tested as a standalone FPGA design. This step does not produce a complete board-level system. Instead, it checks whether the RTL can be accepted by the FPGA toolchain.

The standalone implementation verifies:

- whether the RTL can be synthesized;
- whether the design can pass place-and-route;
- how many FPGA resources it uses;
- what clock frequency the timing report estimates.

This is an important intermediate step because an RTL design that passes simulation may still fail during synthesis or implementation.

2.2 Libero Environment

The standalone implementation was performed using Libero SoC 2025.2. The target device was the BeagleV-Fire compatible PolarFire SoC part:

```
1 MPFS025T-1FCVG484E
```

The main RTL files added to the Libero project were:

```
1 denoise_core_simple.sv
2 denoise_axis_stream_simple.sv
3 denoise_controlled_axis.sv
```

The implementation flow included synthesis, place-and-route, resource analysis, timing analysis, and bitstream generation.

2.3 Synthesis Result

Figure 2 shows the synthesis result in Libero. The design was successfully synthesized with no fatal errors.

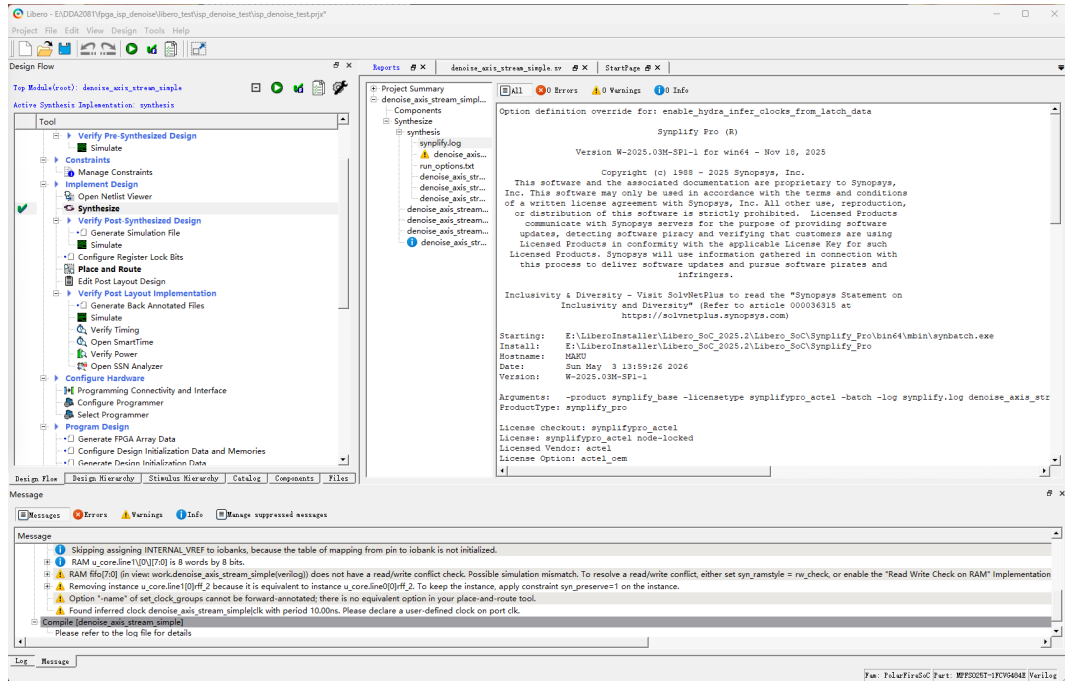


Figure 2: Successful synthesis of the denoise accelerator in Libero SoC.

This result confirms that the accelerator is not only simulation-correct, but also acceptable to the FPGA synthesis tool.

2.4 SmartDesign View

Figure 3 shows the SmartDesign view of the denoise AXI4-Stream module. The module exposes clock and reset pins, along with AXI4-Stream slave and master interfaces.

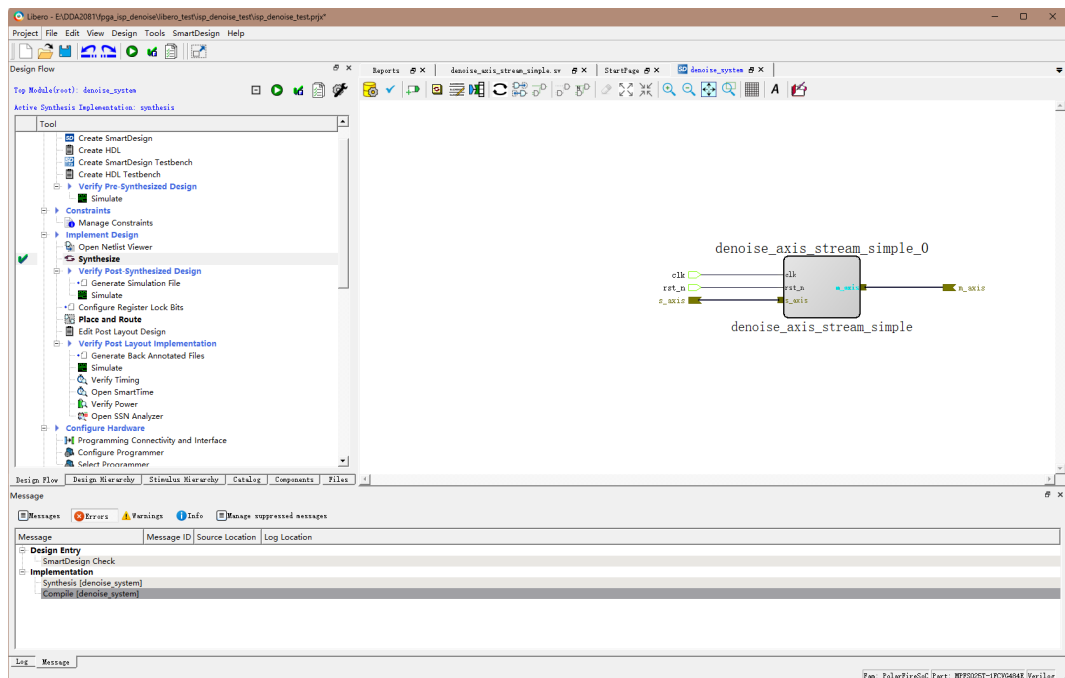


Figure 3: SmartDesign view of the denoise AXI4-Stream module.

This view shows that the accelerator has been packaged as a reusable hardware component, which is required before it can be connected to a larger SoC-level design.

3 Resource Utilization and Timing Analysis

3.1 Resource Utilization

The standalone denoise accelerator used only a small amount of FPGA resources. The resource report is shown in Figure 4.

Module Name	Fabric 4LUT	Fabric DFF	Interface 4LUT	Interface DFF	Single-Ended I/O	uSRAM (64x12)	Chip Globals
Top	151	84	36	36	22	3	1
Primitives	21	0	0	0	22	0	1
denoise_axis_stream_si...	130	84	36	36	0	3	0

Figure 4: Resource utilization report of the standalone denoise accelerator.

The key resource usage is summarized in Table 1.

Table 1: Standalone denoise accelerator resource utilization.

Resource Type	Usage
Fabric 4LUT	151
Fabric DFF	84
Interface 4LUT	36
Interface DFF	36
Single-Ended I/O	22
uSRAM	3
Chip Globals	1

The low resource usage is expected because the current accelerator processes one 8-bit channel and implements only a small 3×3 filter. The uSRAM usage mainly comes from the line-buffer structure. This result suggests that the accelerator should be small enough to integrate into the official BeagleV-Fire gateway.

3.2 Timing Result

The timing report estimated a clock period of approximately:

$$5.080 \text{ ns}$$

This corresponds to:

$$\frac{1}{5.080 \text{ ns}} \approx 196.85 \text{ MHz}$$

Figure 5 shows the timing report.

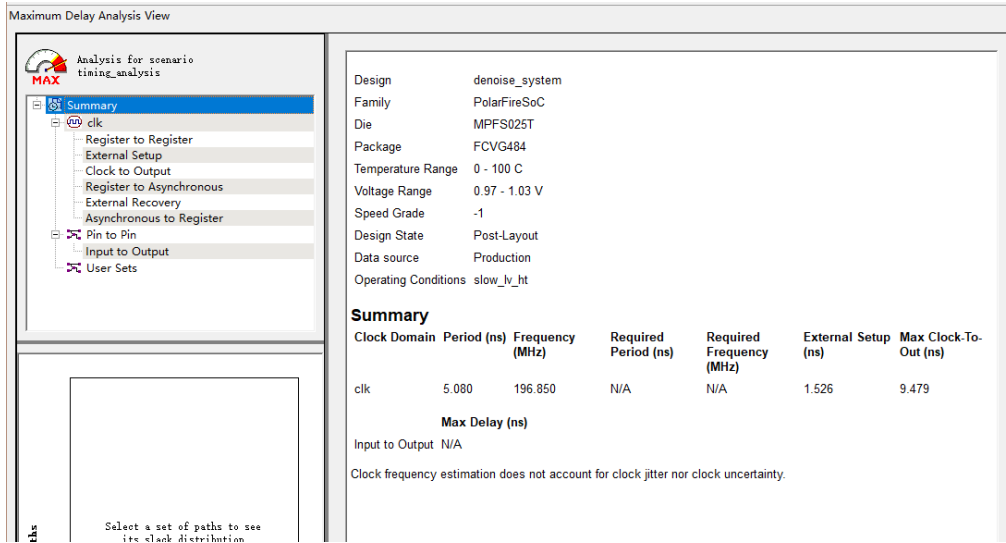


Figure 5: Timing analysis result of the standalone denoise design.

This timing result is promising for a streaming image-processing accelerator. Under continuous input conditions, the denoise pipeline can produce one output pixel per cycle after the initial line-buffer and window-fill latency.

4 Why Standalone Implementation Is Not Enough

Although the standalone Libero result is successful, it cannot be directly used as a complete BeagleV-Fire board image. The standalone design only contains the denoise accelerator logic. It does not include the board-level infrastructure required by Linux or the RISC-V processor subsystem.

A real BeagleV-Fire gateway image must include:

- the RISC-V MSS subsystem;
- DDR memory interface;
- HSS and OpenSBI boot-related configuration;
- Linux device-tree overlay;
- board-level peripheral configuration;
- memory map;
- control path from Linux to FPGA logic;
- data path from DDR to the accelerator and back.

Therefore, the standalone design is useful as a module-level validation step, but the real system must be built by modifying the official BeagleV-Fire gateway. This leads to the next major part of the project: reproducing and patching the official gateway build flow.

5 Official BeagleV-Fire Gateway Build Flow

5.1 Project Directory and Build Command

The official BeagleV-Fire gateway repository was used as the base project. The working directory was:

```
1 E:\DDA2081\beaglev_fire_gateway\gateway
```

The default build command was:

```
1 py build-bitstream.py ./build-options/default.yaml ./boards/mpfs-  
    beaglev-fire.yaml
```

The board configuration specified the PolarFire SoC device and board settings:

```
1 DIE: MPFS025T  
2 PACKAGE: FCVG484  
3 DIE_VOLTAGE: 1.0  
4 PART_RANGE: EXT  
5 SPEED: STD  
6 MSS_DDR: LPDDR4  
7 MSS_DDR_WIDTH: 32
```

Unlike the standalone Libero project, this official flow generates a complete board-level gateway image.

5.2 Toolchain Setup

The Windows build environment required several tools:

- Libero SoC 2025.2;
- Microchip SoftConsole RISC-V toolchain;
- MSYS2 make;
- Python;
- Libero license file;

- FPGA programming tools.

The key environment variables included:

```
1 SC_INSTALL_DIR
2 LIBERO_INSTALL_DIR
3 LM_LICENSE_FILE
4 FPGENPROG
5 PATH
```

Several early build failures were caused by missing tools in PATH. These were fixed by adding Libero Designer tools, SoftConsole RISC-V GCC, and MSYS2 utilities to the Windows PATH.

6 Windows Compatibility Issues and Fixes

Running the official BeagleV-Fire gateway builder under Windows required several patches. The main issues are summarized below.

6.1 Path and Tool Problems

The first group of issues was related to tool discovery and path configuration:

- `libero` was not found;
- `pfsoc_mss` was not found;
- `SC_INSTALL_DIR` was not set;
- the RISC-V toolchain was not in PATH;
- `make` was not found.

These were fixed by correcting the environment variables and PATH.

6.2 Python Output Decoding

The build script originally decoded command output using ASCII. On Windows, some output contained non-ASCII characters, causing decode errors. The fix was to use a Windows-compatible decoding method:

```
1 line.decode("mbcs", errors="replace")
```

This allowed the script to continue even when unexpected characters appeared in tool output.

6.3 HSS and Configuration Generation

The HSS build stage caused several problems, including Kconfig, BOARD, OPENSBI source directory, GENCONFIG, and hss-deflate errors. Some of these problems were related to Windows path handling in the configuration generator.

A manual test showed that the configuration generator worked when Linux-style paths were used. This indicated that the generator did not handle Windows backslash paths reliably. The patched flow therefore pre-generated required configuration files and skipped problematic generation steps when needed.

6.4 MSS PermissionError

The build process also encountered a Windows file permission issue when deleting or refreshing `sources/MSS`. Git pack files could remain locked by another process, causing:

```
1 PermissionError: [WinError 5] Access is denied
```

The successful patch avoided unnecessary deletion of the MSS source directory.

6.5 Libero SCRIPT_ARGS Path Truncation

A major issue came from passing absolute Windows paths through Libero `SCRIPT_ARGS`. Windows drive letters such as `E:` contain a colon, which caused path parsing problems inside the Libero flow.

The fix was to pass relative paths to Libero script arguments whenever possible. The script Tcl path remained absolute, but paths such as MSS component location and HSS image location were generated relative to the Libero working directory.

6.6 Tcl Temporary File Permission Issue

The original `functions.tcl` used a temporary file generation method that caused permission errors on Windows. The fix was to generate temporary files under a controlled project directory:

```
1 E:/DDA2081/beaglev_fire_gateway/gateway/tmp_libero
```

This avoided the Windows temporary-file permission issue.

6.7 Two-Stage Libero Flow

The original Libero Tcl flow could complete place-and-route but fail to continue reliably to later steps such as timing verification and programming-file generation. To solve this, a second-stage script was created:

```
1 continue_after_route.tcl
```

This script reopened the routed Libero project and continued:

```
1 VERIFYTIMING
2 ADD_ENVM
3 export_spi_prog_file
4 configure_spiflash
5 GENERATEPROGRAMMINGFILE
6 export_flashproexpress
7 export_directc
```

This two-stage strategy was essential because the final board-deployable SPI file is generated only after the later programming-file export steps.

6.8 Summary

The main Windows compatibility fixes are summarized in Table 2.

Table 2: Summary of Windows compatibility issues and fixes.

Issue	Fix
Tool not found	Added Libero, SoftConsole, RISC-V GCC, and MSYS2 paths
ASCII decode error	Used Windows-compatible decoding with replacement
HSS generation errors	Pre-generated configuration files and skipped problematic steps
MSS PermissionError	Avoided deleting locked MSS Git directory
SCRIPT_ARGS truncation	Used relative paths instead of absolute Windows paths
Tcl temp file error	Used controlled <code>tmp_libero</code> directory
Flow stopped after route	Added second-stage continuation Tcl script

After these changes, the official default BeagleV-Fire gateway build could complete successfully under Windows.

7 Default Bitstream Generation and Board Validation

7.1 Successful Default Bitstream Export

After applying the Windows patches and two-stage Libero strategy, the official default BeagleV-Fire gateway build completed successfully. The important output file was:

```
1 bitstream/LinuxProgramming/mpfs_bitstream.spi
```

It was backed up as:

```
1 mpfs_bitstream_default_success.spi
```

The file size was:

```
1 2,382,112 bytes
```

The successful log showed that timing constraints were met, FPGA array data was generated, and the SPI master file was exported. Figure 6 shows the successful default bitstream export.

```
Info: Found sNVM configuration file: E:\DDA2081\beaglev_fire_gateway\gateway\work\libero\designer\DEFAULT_3F8EC993DA6CF8E62F7B0B\SNVM.cfg
Info: Found eNVM configuration file: E:\DDA2081\beaglev_fire_gateway\gateway\work\libero\designer\DEFAULT_3F8EC993DA6CF8E62F7B0B\ENVM.cfg
TBBmalloc: skip allocation functions replacement in ucrtbase.dll: unknown prologue for function _msize
Info: Generated dca file E:\DDA2081\beaglev_fire_gateway\gateway\work\libero\designer\DEFAULT_3F8EC993DA6CF8E62F7B0B\DEFAULT_3F8EC993DA6CF8E62F7B0B.dca
Exporting Bitstream File(s)...
Software Version: 2025.2.0.14
Opened 'E:\DDA2081\beaglev_fire_gateway\gateway\work\libero\designer\DEFAULT_3F8EC993DA6CF8E62F7B0B\DEFAULT_3F8EC993DA6CF8E62F7B0B_fp\DEFAULT_3F8EC993DA6CF8E62F7B0B.pro'
PDB file 'E:\DDA2081\beaglev_fire_gateway\gateway\work\libero\designer\DEFAULT_3F8EC993DA6CF8E62F7B0B\DEFAULT_3F8EC993DA6CF8E62F7B0B.pdb' has been loaded successfully.
DESIGN : DEFAULT_3F8EC993DA6CF8E62F7B0B; CHECKSUM : 84D1; PDB_VERSION : 1.9
Load Programming Data(s) Finished : Sun May 03 20:14:42 2026 (Elapsed time 00:00:00)
Info: Programming Interface selected is JTAG.
Info: Design version: 2035
File/Folder 'E:\DDA2081\beaglev_fire_gateway\gateway\bitstream\LinuxProgramming\mpfs_bitstream.spi' will be overwritten.
Successfully exported SPI master file: 'E:\DDA2081\beaglev_fire_gateway\gateway\bitstream\LinuxProgramming\mpfs_bitstream.spi'; file programs Fabric, sNVM and eNVM.
Fabric component bitstream digest: 70af2a8f22a7a23727671b2a610dbf6f188e6e771b6a2eef3667f5f3fc7988d4
eNVM component bitstream digest: 5e66cd9e016e45b956cd6ce5636d19877d4a257ab9ad68cdcd5114a5cf1e5467
sNVM component bitstream digest: 4b1bcaff234b852f6f42e45f7003218570781aca3ed96830eb09fb37ef814b42
Entire bitstream digest: 7e193e2336cf14aca40b8baa5c87498a6c3cla5ff3e1b2479d99489cfa582032
Finished: Sun May 03 20:14:47 2026 (E
PS E:\DDA2081\beaglev_fire_gateway\gateway>
```

Figure 6: Successful export of the default BeagleV-Fire SPI bitstream.

This result confirmed that the patched Windows build flow could generate a complete official BeagleV-Fire gateway image.

7.2 Board Access and File Transfer

The BeagleV-Fire board was accessed through PuTTY serial console using:

- COM3;
- 115200 baud;
- 8 data bits, no parity, 1 stop bit;
- username `beagle`;
- password `temppwd`.

The board IP was checked with:

```
1 hostname -I
```

The usable USB network address was:

```
1 192.168.7.2
```

The generated bitstream was transferred from Windows to the board using SCP:

```
1 scp mpfs_bitstream_default_success.spi beagle@192.168.7.2:/home/  
beagle/
```

On the board, the file size was checked and confirmed to be approximately 2.3 MB.

7.3 Linux-Side Gateway Update

The official board-side update script is:

```
1 /usr/share/beagleboard/gateway/change-gateway.sh
```

The script expects a gateway directory containing:

```
1 LinuxProgramming/mpfs_bitstream.spi  
2 LinuxProgramming/mpfs_dtbo.spi
```

A custom directory was created:

```
1 /home/beagle/my_default_gateway/LinuxProgramming
```

The generated default bitstream was copied into this directory as `mpfs_bitstream.spi`, and the default device-tree overlay was copied as `mpfs_dtbo.spi`. The update command was:

```
1 sudo /usr/share/beagleboard/gateway/change-gateway.sh /home/  
beagle/my_default_gateway
```

The board rebooted after the update. After reboot, the system came back normally. This confirmed that the Windows-generated default bitstream was valid for real board deployment.

8 CAPE / VERILOG_TEMPLATE / APB Integration Investigation

8.1 Why APB First

The long-term target is a DMA-based AXI4-Stream data path. However, this is complex because it requires DDR buffers, DMA configuration, device-tree support, and Linux-side software. Therefore, a safer first integration route is to use an APB-accessible register interface through the official CAPE template.

The goal of this first step is not to process full images. The goal is only to confirm:

RISC-V Linux → APB bridge → custom FPGA register

If this path works, the denoise control signals can later be connected to registers.

8.2 VERILOG_TEMPLATE

The official repository includes:

```
1 sources/FPGA-design/script_support/components/CAPE/  
   VERILOG_TEMPLATE
```

Important files include:

```
1 ADD_CAPE.tcl  
2 HDL/CAPE.v  
3 HDL/apb_ctrl_status.v
```

The CAPE template connects:

```
1 CAPE:APB_TARGET <-> BVF_RISCV_SUBSYSTEM:CAPE_APB_MTARGET
```

This means the official design already provides a route from the RISC-V subsystem to the CAPE APB target.

8.3 APB Register Test

The existing APB test module defines a simple register map:

Table 3: Initial APB register map in the VERILOG_TEMPLATE CAPE design.

Offset	Register	Behavior
0x00	CONTROL_0	Read-only 0xdeadbeef
0x10	CONTROL_1	Read/write control value
0x20	STATUS	Read status input

The 0xdeadbeef register is useful as a smoke test. If Linux can read this value, the CPU-to-FPGA APB path is working.

8.4 APB Test Build

A new build configuration was created:

```
1 build-options/denoise_apb_test.yaml
```

It enabled:

```
1 CAPE_OPTION:VERILOG_TEMPLATE
2 M2_OPTION:NONE
```

The APB test build generated a design named:

```
1 DENOISE_APB_TEST_3F8EC993DA6CF
```

The generated bitstream was backed up as:

```
1 mpfs_bitstream_denoise_apb_test.spi
```

The file size was approximately:

```
1 2,380,032 bytes
```

This showed that the patched Windows build flow could also generate a modified gateway configuration.

8.5 Experimental Deployment

The APB test bitstream was copied to:

```
1 /home/beagle/my_denoise_apb_gateway/LinuxProgramming
```

and renamed as:

```
1 mpfs_bitstream.spi
```

Then the official update script was executed:

```
1 sudo /usr/share/beagleboard/gateway/change-gateway.sh /home/
   beagle/my_denoise_apb_gateway
```

Figure 7 shows the board-side update process.

```
beagle@BeagleV:~$ sudo /usr/share/beagleboard/gateway/change-gateway.sh /home/beagle/my_denoise_apb_gateway
[sudo] password for beagle:
Sorry, try again.
[sudo] password for beagle:
Changing gateway.
'/home/beagle/my_denoise_apb_gateway/LinuxProgramming/mpfs_dtbo.spi' -> '/lib/firmware/mpfs_dtbo.spi'
'/home/beagle/my_denoise_apb_gateway/LinuxProgramming/mpfs_bitstream.spi' -> '/lib/firmware/mpfs_bitstream.spi'

=====
                        FPGA Gateway Update
=====

Please ensure that the mpfs_bitstream.spi file containing the gateway
update has been copied to directory /lib/firmware.

!!! This will take a couple of minutes. !!!

Give the system a few minutes to reboot itself
after Linux has shutdown.

=====

Erased 65536 bytes from address 0x00000000 in flash
Writing mpfs_dtbo.spi to /dev/mtd0
Triggering FPGA Gateway Update (/sys/kernel/debug/fpga/microchip_exec_update)

Broadcast message from root@BeagleV on pts/1 (Fri 2024-05-24 09:20:55 UTC):

The system will reboot now!
```

Figure 7: Linux-side gateway update process using `change-gateway.sh`.

The log shows that the script copied the bitstream and device-tree overlay to `/lib/firmware`, wrote the overlay to flash, triggered the FPGA update, and rebooted the system.

9 Deployment Issue and Risk Analysis

9.1 Observed Issue

After deploying the experimental APB-test gateway image, the board did not enumerate as a USB serial device or USB network device after reboot. The previous COM port did not appear, and the USB network interface was also missing. Ethernet DHCP and ARP checks did not find a reachable board IP.

The board still showed LED activity, so the issue was not treated as immediate physical hardware damage. A more likely explanation is that the system did not boot far enough to expose the usual Linux USB serial or network functions.

9.2 Likely Cause

The most likely cause is a system-level gateway or boot-chain mismatch. A key risk is that the APB-test bitstream may not have been deployed with a matching device-tree overlay.

The Linux-side update flow expects:

- 1 `mpfs_bitstream.spi`
- 2 `mpfs_dtbo.spi`

to describe the same hardware configuration. If the bitstream changes the FPGA design but the device-tree overlay does not match it, Linux or the boot flow may fail.

In this experiment, a default device-tree overlay was reused during APB-test deployment. This was convenient for quick testing, but it was not the safest approach.

9.3 Recovery Options

If Linux is still reachable, the default gateway can be restored using:

```
1 sudo /usr/share/beagleboard/gateway/change-gateway.sh /usr/  
share/beagleboard/gateway/default
```

If Linux is not reachable through USB serial, USB network, or Ethernet, the Linux-side update method cannot be used. In that case, recovery requires Microchip FlashPro Express with a FlashPro5 or FlashPro6 JTAG programmer and a known-good default .job file.

9.4 Lessons Learned

This issue clarified several important points:

- A successful bitstream build does not guarantee safe board deployment.
- The bitstream and device-tree overlay should be generated and deployed as a matched pair.
- Experimental gateway updates need a reliable recovery method.
- APB register testing is still the correct first step, but deployment should use matched LinuxProgramming outputs.

This was an important practical lesson in FPGA SoC development.

10 Revised Integration Plan

Based on the results of this stage, the next integration attempt should be more conservative.

10.1 Step 1: Restore Default Gateway

The first priority is to restore the board to a known-good default gateway state. If Linux access is available, the official default gateway can be restored using `change-gateway.sh`. If Linux is not available, FlashPro/JTAG recovery is required.

10.2 Step 2: Use Matched Linux Programming Files

Future deployment should use the complete matched output from the same build:

```
1 mpfs_bitstream.spi
2 mpfs_dtbo.spi
3 mpfs_bitstream_spi.digest
```

Reusing a device-tree overlay from another gateway configuration should be avoided.

10.3 Step 3: Verify APB Register Access

Before inserting the denoise accelerator, a simple APB register test should be completed. The expected test sequence is:

- read 0xdeadbeef from offset 0x00;
- write a value to offset 0x10;
- read back the same value from offset 0x10;
- read status from offset 0x20.

This confirms the CPU-to-FPGA register path.

10.4 Step 4: Connect Denoise Control Signals

After APB access is verified, the `denoise_controlled.axis` control signals can be mapped to registers. A possible register map is:

Table 4: Proposed register mapping for denoise control.

Offset	Register	Description
0x00	ID	Read-only test value
0x04	CONTROL	Bit 0: start
0x08	STATUS	Bit 0: busy, Bit 1: done
0x0C	WIDTH	Image width
0x10	HEIGHT	Image height
0x14	DEBUG	Output counter or debug value

10.5 Step 5: Add Data Movement

The final step is to add real image data movement. A small APB-based test can be used first for tiny inputs. After that, a DMA or AXI4-Stream path can be added for full image buffers.

11 Conclusion

This final stage moved the project from RTL simulation to FPGA implementation and official BeagleV-Fire gateway integration.

First, the verified denoise accelerator was implemented as a standalone Libero design. Synthesis and place-and-route completed successfully. The resource utilization was low, and the timing report estimated a frequency of approximately 196.85 MHz. This confirmed that the accelerator is suitable for FPGA implementation.

Second, the project moved beyond standalone RTL and reproduced the official BeagleV-Fire gateway build flow under Windows. Several compatibility issues were fixed, including tool path problems, HSS generation issues, Windows path handling, Libero script argument truncation, Tcl temporary file permission problems, and post-route flow continuation. After these fixes, the default official gateway was successfully generated as a LinuxProgramming SPI bitstream.

Third, the generated default bitstream was transferred to the BeagleV-Fire board and deployed through the official Linux-side update mechanism. The board rebooted successfully after the default update, proving that the patched Windows build flow could produce a usable board-level image.

Finally, the CAPE / VERILOG_TEMPLATE / APB path was investigated as a safer first integration route for custom logic. An APB-test gateway image was generated, but its deployment exposed a system-level risk. The likely issue was a mismatch between the experimental bitstream and the deployed device-tree overlay. This clarified the importance of using matched LinuxProgramming outputs and maintaining a recovery path through FlashPro/JTAG.

The next stage should first restore the board to a known-good state, then repeat the APB test using matched bitstream and device-tree overlay files. After APB register access is verified from Linux, the `denoise_controlled_axis` module can be connected to software-visible control registers. Only after this register-level path is stable should full AXI4-Stream or DMA-based image movement be attempted.